

锦标赛

将所有人排序之后，如果两个相邻的人之间的差超过了 k ，那么低的人就无法战胜高的人。

所以从大到小，找到第一段差超过 k 的，那么后面的人没有办法赢。前面的人可以从前往后依次打过去，第二大战胜第一大，第三大战胜第二大，这样下去，都能获胜。

树上求和

对于每个颜色，我们统计有多少对点包含这个颜色。这个相当于求出删去这种颜色的所有点之后，统计每个连通块里路径总数。

最直接的做法就是用类似于虚树的做法。这里提供一个更加简单的做法。首先考虑一个单独的点，那么它删除之后，连通块就是各个子树。如果删除了更多的点，对于每个点，需要在它往上第一个和它颜色相同的点的子树上删除。

这样就可以做到 $O(n)$ 。

最小公倍数

首先不妨设 a_i, b_i 两两不同。考虑枚举集合中 b_i 最大的元素，将所有 $b_j < b_i$ 的元素，按 a_j 从小到大排序，不妨记作 $a'_1, a'_2, \dots, a'_{i-1}$ ，那么会有 2^{i-1} 个子集，它们的最大值是 a'_k 。

考虑将 a_i 加入之后，那么所有最大值小于 a_i 的子集，它们的最大值变成了 a_i ，剩下的子集不变。将 a_i 加入到上面的有序序列之后，所有比 a_i 大的元素，它们的排名会加一，对应的子集个数会翻倍。

问题等价于区间乘 2，区间求和，可以用线段树解决。

回文子序列

首先考虑 $dp_{l,r}$ 表示 $[l,r]$ 这一段，其中 $a[l] = a[r]$ ，最长的回文子序列和对应的个数是多少。

令 $pre_{i,c}$ 表示 i 前一个 c 出现的位置， $suf_{i,c}$ 表示 i 后一个 c 出现的位置。因为要求本质不同，所以我们可以枚举子序列中下一个字符 c ，然后从 $dp_{suf_{l,c}, pre_{r,c}}$ 转移到 $dp_{l,r}$ 位置，这样就能保证不重复。这样的时间复杂度是 $O(n^3)$ 的。

考虑固定了 l ，从小到大枚举 r 的时候，每次从 r 转移到 $r+1$ 的时候， pre_{r+1, a_r} 会变为 r ，其他的元素都不会变。

令 $f_c = dp_{suf_{l,c}, pre_{r,c}}$ ，那么只有 f_{a_r} 会变大，其他都不会变。而我们要快速回答 f 中最大值和等于最大值的元素的和，这个可以简单 $O(1)$ 维护。

总的时间复杂度 $O(n^2)$ 。

代码源